

Identificação do Plano de Ensino

Unidade	Rio do Sul				
Curso	Técnico em Informática para Internet				
Unidade Curricular	Testes de Front-End				
Carga Horária da UC	40:00	Ano / Semestre	2026/02	Turno:	Manhã
Turma	T TIIN 2026/1 M1	Nº de Situações de Aprendizagem			2
Docente:	Gelvazio Camargo				

Informações da Unidade Curricular e Curso**Competência Geral do Curso**

Produzir interfaces e desenvolver sistemas para internet, de acordo com metodologia e padrões de qualidade, usabilidade, interatividade, robustez, acessibilidade e segurança da informação.

Função

- Função 1: Produzir Interfaces para internet, de acordo com metodologia e padrões de qualidade, usabilidade, interatividade, robustez, acessibilidade e segurança da informação.
- Função 2: Desenvolver sistemas para internet, de acordo com metodologia e padrões de qualidade, usabilidade, interatividade, robustez, acessibilidade e segurança da informação.

Objetivo Geral da Unidade Curricular

- Objetivo Geral: Propiciar desenvolvimento de capacidades básicas e socioemocionais para codificação de interfaces baseadas em UX e UI em aplicações web, considerando as necessidades do usuário.

Situação de Aprendizagem: SA BLOCO 02 — Desenvolvimento de Pipeline CI/CD e Dashboard**Capacidades Básicas / Capacidades Técnicas**

- Executar testes de funcionamento da interface para web
- Executar planos de testes de interface para web
- Elaborar plano de testes de interface para web
- Documentar resultados de testes de funcionamento da interface para web

Objetos de Conhecimento

- Técnicas de testes: Teste funcional caixa preta. Teste estrutural caixa branca.
- Tipos de testes: Funcionalidade. Usabilidade. Confiabilidade. Desempenho. Manutenibilidade.
- Processo fundamental de teste: Planejamento. Desenho dos Testes. Execução dos Testes. Monitoração e Controle. Avaliação dos Resultados.
- Planejamento de testes client-side: Análise do documento de requisitos. Plano de testes. Suíte de testes. Casos de testes.
- Conceitos fundamentais: Verificação. Validação.
- Automação de Testes: Definição. Frameworks. Aplicação. Interação com equipe de testes.
- Autogestão: Responsabilidade.

Capacidades Socioemocionais

- Valorizar novos fatos, ideias e opiniões diferentes para resolução de problemas pertinentes as atividades sob a sua responsabilidade
- Fundamentar escolhas e decisões no exame de fatos, contextos, possibilidades, desafios e problemáticas de diferentes naturezas à luz de referenciais técnicos, legais, normativos e institucionais

Estratégia de Aprendizagem Desafiadora

(X) Situação-Problema

() Estudo de Caso

() Projeto

() Pesquisa Aplicada

Contextualização

Uma equipe de desenvolvimento trabalha em projeto React. Após estruturar plano de testes (SA-BLOCO-01) e escrever primeira batida de testes (Blocos 01-02), percebem que:

- Testes são executados manualmente antes de cada deploy (processo lento e propenso a erro)
- Não há visibilidade de cobertura de testes (ninguém sabe qual % do código está testado)
- PRs são mergeadas sem validação de qualidade (testes podem estar falhando)
- Quando um bug chega a produção, ninguém consegue rastrear por que os testes não pegaram

A equipe de DevOps foi convidada para implementar ****pipeline de integração contínua**** que execute testes automaticamente e crie ****dashboard de cobertura**** mostrando saúde do projeto.

Desafio

Você recebe repositório GitHub com:

- Código-fonte React de aplicação (src/)
- Testes Vitest já escritos (test/)
- Testes E2E com Cypress (cypress/e2e/)

Sua tarefa é criar ****pipeline completo de CI/CD**** que:

1. ****Automação em GitHub Actions:****

- Criar workflow que roda em cada push e PR
- Executar npm test (Vitest com coverage)

- Executar npm run e2e (Cypress)
- Bloquear merge se testes falharem
- Gerar relatório de cobertura

2. **Métricas de Qualidade:**

- Coletar relatório de cobertura (line, branch, function, statement)
- Calcular defect density (bugs/1000 linhas)
- Rastrear flakiness rate (% testes intermitentes)
- Comparar cobertura antes/depois

3. **Visibilidade (Dashboard/Badges):**

- Badge de status de testes no README (pass/fail)
- Badge de cobertura (%) no README
- Histórico de cobertura (semana anterior vs atual)
- Relatório HTML de cobertura interativo
- Identificação visual de arquivos com baixa cobertura

4. **Integração com GitHub:**

- Comentário automático em PR: "Cobertura: 75% 78% "
- Bloquear merge se cobertura cair abaixo de 70%
- Status check no PR indicando quais testes passaram/falharam

5. **Documentação de Pipeline:**

- Explicação clara do workflow (.github/workflows/test.yml)
- Instruções para devs: "Como rodar testes localmente"
- Instruções para novatos: "Como contribuir mantendo qualidade"
- Troubleshooting: "Teste está falhando, o que fazer?"

6. **Apresentação do Pipeline:**

- Demonstração ao vivo do workflow em ação
- Explicação do fluxo (push testes automáticos relatório)
- Discussão sobre como isso previne bugs em produção
- Sugestões de otimizações futuras

Resultados Esperados

****Portfólio Técnico Profissional contendo:****

****Arquivos de Configuração:****

- .github/workflows/test.yml - Workflow completo de CI/CD
- .github/workflows/coverage.yml - Workflow de relatório de cobertura (opcional)
- vitest.config.js - Configuração Vitest com coverage
- cypress.config.js - Configuração Cypress

****Documentação:****

- TESTING.md - Guia completo de testes (como rodar, como escrever, how to debug)
- CONTRIBUTING.md - Guia para contribuidores (requisitos de qualidade)
- CI-CD-WORKFLOW.md - Explicação do pipeline (para who wants to understand)

****Relatórios:****

- Relatório HTML de cobertura (dist/coverage/index.html)
- Histórico de cobertura (últimas 10 execuções)
- Status badges funcionando no README

****README Atualizado:****

...

Meu Projeto

[![Tests](https://github.com/user/repo/workflows/Tests/badge.svg)](...)

[![Coverage](https://img.shields.io/codecov/c/github/user/repo)](...)

[![License](https://img.shields.io/badge/license-MIT-blue.svg)](...)

Testing

Rodar testes localmente: `npm test`

Rodar E2E: `npm run cypress`

Ver cobertura: `npm test -- --coverage`

...

****Demonstração Prática:****

- Fazer push de código com teste falhando
- Mostrar workflow rodando automaticamente
- Mostrar GitHub bloqueando merge
- Corrigir teste, fazer novo push
- Mostrar workflow passando e merge liberado

Plano Aula: BLOCO-02-VITEST-E-TESTING-LIBRARY - 20H

CH alocada: 20:00

Capacidades a serem trabalhadas

- Executar testes de funcionamento da interface para web
- Documentar resultados de testes de funcionamento da interface para web
- Executar planos de testes de interface para web
- Elaborar plano de testes de interface para web

Conhecimentos Relacionados

- Autogestão: Responsabilidade.
- Automação de Testes: Definição. Frameworks. Aplicação. Interação com equipe de testes.
- Técnicas de testes: Teste funcional caixa preta . Teste estrutural caixa branca .
- Tipos de testes: Funcionalidade. Usabilidade. Confiabilidade. Desempenho. Manutenibilidade.
- Conceitos fundamentais: Verificação. Validação.
- Planejamento de testes client-side: Análise do documento de requisitos. Plano de testes. Suíte de testes. Casos de testes.
- Processo fundamental de teste: Planejamento. Desenho dos Testes. Execução dos Testes. Monitoração e Controle. Avaliação dos Resultados.

Estratégias de Ensino

- Gamificação
- Pannel Temático

	- Seminário - Workshop - Ensaio Tecnológico - Dinâmica de Grupo - Trabalho em Grupo - Atividade Prática - Exposição Dialogada
Recursos e Ambientes Pedagógicos	sala de aula, projetor, notebooks, computadores
Critérios de Avaliação (como vou avaliar)	- Instala e configura Vitest e Testing Library - Estrutura teste com describe/it/expect corretamente - Utiliza matchers apropriados por tipo de dado - Cria mocks com vi.fn() funcionalmente - Utiliza spies com vi.spyOn() e verifica chamadas - Implementa fixtures (beforeEach, afterEach) - Executa testes em watch mode e interpreta saída - Lê e interpreta relatório de coverage - Utiliza queries apropriadas (getByRole prioritário) - Simula interações realistas com userEvent - Testa componentes com múltiplos estados - Testa operações assíncronas com waitFor - Debuga testes com console.log e screen.debug() - Aplica padrão AAA em testes - Escreve testes que validam comportamento, não implementação
Instrumentos de Avaliação da Aprendizagem	- Atividade prática de instalação e primeiro teste funcionando - Exercício de cobertura completa com múltiplos matchers - Projeto de mocks e spies em contexto real - Atividade prática de Testing Library com 5+ componentes - Trabalho em grupo: suite de testes para aplicação autêntica - Code review de testes escritos por colega - Apresentação: como foram tomadas decisões de teste

Situação de Aprendizagem: SA BLOCO 01 — Estruturação e Documentação Profissional de**Capacidades Básicas / Capacidades Técnicas**

- Executar testes de funcionamento da interface para web
- Executar planos de testes de interface para web
- Elaborar plano de testes de interface para web
- Documentar resultados de testes de funcionamento da interface para web

Objetos de Conhecimento

- Técnicas de testes: Teste funcional caixa preta. Teste estrutural caixa branca.
- Tipos de testes: Funcionalidade. Usabilidade. Confiabilidade. Desempenho. Manutenibilidade.
- Processo fundamental de teste: Planejamento. Desenho dos Testes. Execução dos Testes. Monitoração e Controle. Avaliação dos Resultados.
- Planejamento de testes client-side: Análise do documento de requisitos. Plano de testes. Suíte de testes. Casos de testes.
- Conceitos fundamentais: Verificação. Validação.

- Automação de Testes: Definição. Frameworks. Aplicação. Interação com equipe de testes.
- Autogestão: Responsabilidade.

Capacidades Socioemocionais

- Valorizar novos fatos, ideias e opiniões diferentes para resolução de problemas pertinentes as atividades sob a sua responsabilidade
- Fundamentar escolhas e decisões no exame de fatos, contextos, possibilidades, desafios e problemáticas de diferentes naturezas à luz de referenciais técnicos, legais, normativos e institucionais

Estratégia de Aprendizagem Desafiadora☒ Situação-Problema☐ Estudo de Caso☐ Projeto☐ Pesquisa Aplicada**Contextualização**

Uma startup de tecnologia está desenvolvendo aplicação de e-commerce em React. A equipe de desenvolvimento precisa estabelecer estratégia de testes antes de iniciar implementação de features críticas (autenticação, carrinho, checkout). Atualmente não há documentação de testes, não há definição de proporção U/I/E2E, não há rastreabilidade de requisitos para testes, e ninguém sabe qual é o plano de qualidade do projeto.

A equipe de qualidade foi contratada para estruturar tudo: criar plano formal, definir estratégia de teste, estimar esforço, documentar casos de teste, garantir rastreabilidade. O resultado será base para que desenvolvedores saibam exatamente o que, quanto e como testar.

Desafio

Você recebe especificação técnica de 3 features críticas do e-commerce:

1. **Autenticação com Email/Senha**
2. **Carrinho de Compras**
3. **Checkout e Pagamento**

Sua tarefa é criar **plano profissional e documentação de testes** que permita que qualquer desenvolvedor saiba:

- O que será testado (escopo)
- Como será testado (estratégia: U/I/E2E)
- Quanto será testado (estimativa de tempo)
- Quando será testado (cronograma)
- Quais são os riscos identificados

Resultados Esperados

1. **Plano de Testes Formal (1-2 páginas):**
 - Informações gerais (projeto, datas, responsáveis)
 - Escopo (o que será/não será testado)
 - Estratégia (tipos, proporção, ferramentas, cronograma)
 - Métricas esperadas (cobertura, defect density)
 - Riscos identificados e plano B
2. **Casos de Teste Documentados (TC-001 até TC-XXX):**
 - Para cada feature, mínimo 5 casos de teste bem-formados
 - Formato padrão: ID, Título, Pré-requisitos, Passos, Resultado Esperado
 - Abrangendo happy path e edge cases (para autenticação: login válido, senha errada, email não existe, etc)

- Especificar tipo de teste: U (unitário), I (integração), E (E2E)
- 3. ****Matriz de Rastreabilidade:****
 - Cada requisito mapeado para 1+ caso de teste
 - Cada caso de teste mapeado para 1+ requisito
 - Tabela: Requisito TC Tipo Teste Status
- 4. ****Estimativa de Tempo:****
 - Análise realista: quanto tempo cada caso de teste levará
 - Distribuição de esforço entre U/I/E2E
 - Timeline de execução por feature
 - Identificação de críticos vs nice-to-have
- 5. ****Documento de Análise de Risco:****
 - Riscos identificados (ex: "feature de pagamento tem risco alto")
 - Probabilidade e impacto
 - Plano de mitigação (ex: aumentar cobertura de testes)
- 6. ****Apresentação para Stakeholders:****
 - Explicação clara do plano para equipe de desenvolvimento
 - Responder: "Quanto testes precisamos escrever?"
 - Responder: "Quanto tempo vai levar?"
 - Responder: "Como sabemos que testamos o suficiente?"

Plano Aula: BLOCO-01-FUNDAMENTOS-E-PLANEJAMENTO - 20H

CH alocada: 20:00

Capacidades a serem trabalhadas

- Executar testes de funcionamento da interface para web
- Documentar resultados de testes de funcionamento da interface para web
- Executar planos de testes de interface para web
- Elaborar plano de testes de interface para web

Conhecimentos Relacionados

- Autogestão: Responsabilidade.
- Automação de Testes: Definição. Frameworks. Aplicação. Interação com equipe de testes.
- Técnicas de testes: Teste funcional caixa preta . Teste estrutural caixa branca .
- Tipos de testes: Funcionalidade. Usabilidade. Confiabilidade. Desempenho. Manutenibilidade.
- Conceitos fundamentais: Verificação. Validação.
- Planejamento de testes client-side: Análise do documento de requisitos. Plano de testes. Suíte de testes. Casos de testes.
- Processo fundamental de teste: Planejamento. Desenho dos Testes. Execução dos Testes. Monitoração e Controle. Avaliação dos Resultados.

Estratégias de Ensino	- Gamificação - Painel Temático - Seminário - Workshop - Ensaio Tecnológico - Dinâmica de Grupo - Trabalho em Grupo - Atividade Prática - Exposição Dialogada
Recursos e Ambientes Pedagógicos	sala de aula, projetor, notebook, computador
Critérios de Avaliação (como vou avaliar)	- Diferencia tipos de testes (unitário, integração, E2E) corretamente - Compreende proporção ideal da pirâmide de testes - Identifica quando cada tipo de teste é apropriado - Entende diferença entre verificação e validação - Reconhece importância de automação vs manual - Estrutura plano de testes com escopo, estratégia, riscos - Escreve casos de teste bem-formados (padrão ISO) - Estabelece rastreabilidade requisito teste - Estima tempo de testes realisticamente - Documenta testes profissionalmente - Comunica achados sobre qualidade claramente - Reconhece responsabilidade em qualidade
Instrumentos de Avaliação da Aprendizagem	- Atividade prática de classificação de testes + discussão - Exercício de desenho de pirâmide para 2 projetos reais - Trabalho em grupo: estruturação de plano de testes completo - Apresentação de casos de teste bem-formados - Projeto: plano de testes para feature autêntica - Discussão facilitada sobre decisões de estratégia